

本科实验报告

课程名称:	计算机网络
姓 名:	刘仁钦
学 院:	计算机科学与技术学院
专 业:	计算机科学与技术
学 号:	3230106230
指导教师:	韩劲松

2025 年 12 月 25 日

目录

一、 实验目的	4
二、 实验内容	4
三、 主要仪器设备	5
四、 操作方法与实验步骤	5
1. 协议设计	5
1.1. 报文格式	5
1.2. 协议类型定义	5
1.3. 协议交互流程	5
2. 客户端实现	5
2.1. <code>SendPacket</code> 函数	6
2.2. <code>ServerConnect</code> 函数	6
2.3. <code>GetTime</code> / <code>GetName</code> / <code>ClientList</code> 函数	6
2.4. <code>SendMessage</code> 函数	7
2.5. <code>Recieve</code> 线程函数	7
3. 服务端实现	8
3.1. <code>BuildServer</code> 函数	8
3.2. <code>Recieve</code> 线程函数（服务端）	9
3.3. 并发控制	10
4. 编译与运行	10
五、 实验结果与分析	11
1. 协议报文格式	11
2. 服务器启动	11
3. 客户端启动	11
4. 连接功能测试	12
5. 获取时间功能测试	12
6. 获取主机名功能测试	13
7. 获取客户端列表功能测试	13
8. 消息转发功能测试	14
9. 断开连接功能测试	15
10. 并发性能测试	15
10.1. 测试 1: 单客户端连续请求	15

10.2. 测试 2: 多客户端并发请求	17
六、 思考题	19
1. 问题一	19
2. 问题二	19
3. 问题三	20
4. 问题四	20
5. 问题五	21
七、 讨论、心得	21
1. 主要收获	21
2. 遇到的困难与解决	22
八、 附录	22
1. client/client.cpp	22
2. client/func.cpp	24
3. server/server.cpp	30

浙江大学实验报告

实验项目名称: _____ Lab6 基于 Socket 接口实现自定义协议通信 _____

学生姓名: _____ 刘仁钦 _____ 专业: _____ 计算机科学与技术 _____ 学号: _____ 3230106230 _____

一、实验目的

- 学习如何设计网络应用协议
- 掌握 Socket 编程接口编写基本的网络应用软件

二、实验内容

- 根据自定义的协议规范, 使用 Socket 编程接口编写基本的网络应用软件。
- 掌握 C 语言形式的 Socket 编程接口用法, 能够正确发送和接收网络数据包
- 开发一个客户端, 实现人机交互界面和与服务器的通信
- 开发一个服务端, 实现并发处理多个客户端的请求
- 程序界面不做要求, 使用命令行或最简单的窗体即可
- 功能要求如下:
 1. 运输层协议采用 TCP
 2. 客户端采用交互菜单形式, 用户可以选择以下功能
 - a. 连接: 请求连接到指定地址和端口的服务端
 - b. 断开连接: 断开与服务端的连接
 - c. 获取时间: 请求服务端给出当前时间
 - d. 获取名字: 请求服务端给出其机器的名称
 - e. 活动连接列表: 请求服务端给出当前连接的所有客户端信息 (编号、IP 地址、端口等)
 - f. 发消息: 请求服务端把消息转发给对应编号的客户端, 该客户端收到后显示在屏幕上
 - g. 退出: 断开连接并退出客户端程序
 3. 服务端接收到客户端请求后, 根据客户端传过来的指令完成特定任务:
 - a. 向客户端传送服务端所在机器的当前时间
 - b. 向客户端传送服务端所在机器的名称
 - c. 向客户端传送当前连接的所有客户端信息
 - d. 将某客户端发送过来的内容转发给指定编号的其他客户端
 - e. 采用异步多线程编程模式, 正确处理多个客户端同时连接, 同时发送消息的情况
- 根据上述功能要求, 设计一个客户端和服务端之间的应用通信协议

三、主要仪器设备

- 联网的 PC 机
- Linux 虚拟机

四、操作方法与实验步骤

本次实验的主要任务是基于 TCP 协议实现一个客户端-服务器通信系统。实验分为协议设计、客户端实现、服务端实现和测试运行四个阶段。

1. 协议设计

根据实验要求，我们设计了一个简单的应用层协议：

1.1. 报文格式

报文格式采用“类型字节 + 长度 + 负载”的结构：

- 首字节（1 字节）：表示报文类型
- 长度字段（2 字节）：表示负载数据的长度（大端序）
- 后续字节（可变长）：ASCII 格式的负载数据

1.2. 协议类型定义

```
1 constexpr char INVALID = 0;    // 无效请求
2 constexpr char CONNECT = 1;    // 连接请求
3 constexpr char SIGNAL = 2;     // 消息通知
4 constexpr char TIME = 3;       // 时间请求
5 constexpr char NAME = 4;       // 主机名请求
6 constexpr char LIST = 5;       // 客户端列表请求
7 constexpr char MESSAGE = 6;    // 消息转发请求
8 constexpr char DISCONNECT = 7; // 断开连接
```

cpp

1.3. 协议交互流程

1. **CONNECT**: 客户端发送空负载；服务器分配 ID 并返回（如 "1"）
2. **TIME**: 客户端请求时间；服务器返回时间字符串（如 "Wed Dec 25 13:15:30 2025"）
3. **NAME**: 客户端请求主机名；服务器返回主机名字符串
4. **LIST**: 客户端请求在线列表；服务器返回 "id1\$id2\$id3..."
5. **MESSAGE**: 客户端发送 "目标id\$消息内容"；服务器转发给目标（作为 SIGNAL）
6. **SIGNAL**: 服务器向目标客户端转发消息 "源id\$消息内容"
7. **DISCONNECT**: 客户端主动断开连接

2. 客户端实现

客户端主要实现在 `client/func.cpp` 中，包含以下核心函数：

2.1. SendPacket 函数

组装报文并发送:

```
1  bool SendPacket(char type, const std::string& payload = "") { cpp
2      if (!connected || BaseSock < 0) return false;
3
4      uint16_t payload_len = payload.size();
5      std::string packet;
6      packet.reserve(1 + 2 + payload_len);
7      packet.push_back(type);                // 类型(1字节)
8      packet.push_back((payload_len >> 8) & 0xFF); // 长度高字节
9      packet.push_back(payload_len & 0xFF);      // 长度低字节
10     packet.append(payload);                  // 负载数据
11
12     ssize_t sent = send(BaseSock, packet.data(), packet.size(), 0);
13     return sent == (ssize_t)packet.size();
14 }
```

2.2. ServerConnect 函数

建立与服务器的连接:

1. 创建 TCP socket: `socket(AF_INET, SOCK_STREAM, 0)`
2. 提示用户输入服务器 IP 和端口
3. 配置 `sockaddr_in` 结构并调用 `connect()`
4. 连接成功后创建接收线程 `Recieve` 并分离 (`pthread_detach`)

关键代码片段:

```
1  int sock = socket(AF_INET, SOCK_STREAM, 0); cpp
2  Addr.sin_family = AF_INET;
3  Addr.sin_port = htons(port);
4  inet_pton(AF_INET, ip.c_str(), &Addr.sin_addr);
5  connect(sock, reinterpret_cast<sockaddr*>(&Addr), sizeof(Addr));
6  SendPacket(CONNECT, ""); // 发送连接请求
7  pthread_create(&thread, nullptr, Recieve, nullptr);
8  pthread_detach(thread);
```

2.3. GetTime / GetName / ClientList 函数

这些函数只需调用 `SendPacket` 发送对应类型的请求:

```
1  void GetTime() { cpp
2      if (!connected) { /* 错误提示 */ return; }
```

```

3     if (SendPacket(TIME, ""))
4         std::cout << "[SYS] Request Sending Success" << std::endl;
5 }

```

2.4. SendMessage 函数

发送消息给其他客户端：

1. 提示用户输入目标客户端 ID
2. 提示用户输入消息内容（支持一行文本）
3. 构造负载格式： "目标id\$消息内容"
4. 发送 MESSAGE 类型报文

关键实现：

```

1 std::string id, msg;
2 std::cin >> id;
3 std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\\n');
4 std::getline(std::cin, msg);
5 std::string payload = id + "$" + msg;
6 SendPacket(MESSAGE, payload);

```

cpp

2.5. Recieve 线程函数

后台循环接收服务器消息：

```

1 void* Recieve(void* lpParameter) {
2     char buffer[MAXBUF] = {0};
3     std::string recv_buffer;
4     while (connected) {
5         ssize_t len = recv(BaseSock, buffer, sizeof(buffer), 0);
6         if (len <= 0) { /* 断开处理 */ break; }
7
8         recv_buffer.append(buffer, len);
9         while (recv_buffer.size() >= 3) {
10             char type = recv_buffer[0];
11             uint16_t payload_len = ((uint8_t)recv_buffer[1] << 8) |
                (uint8_t)recv_buffer[2];
12             if (recv_buffer.size() < 3 + payload_len) break;
13
14             std::string payload = recv_buffer.substr(3, payload_len);
15             recv_buffer.erase(0, 3 + payload_len);
16
17             switch (type) {
18                 case CONNECT: /* 显示分配的ID */

```

cpp

```

19         std::cout << "[SYS] Your Client ID is " << payload <<
           std::endl;
20         break;
21         case TIME: /* 显示时间 */
22             std::cout << "[SYS] Current Time is: " << payload <<
           std::endl;
23             break;
24         // ... 其他类型处理
25     }
26 }
27 }
28 return nullptr;
29 }

```

3. 服务端实现

服务端主要实现在 `server/server.cpp` 中。

3.1. BuildServer 函数

初始化并启动服务器：

1. 创建监听 socket
2. 配置地址结构（端口设为学号后四位：6230）
3. 绑定地址： `bind()`
4. 开始监听： `listen()`
5. 循环接受连接： `accept()`
6. 为每个连接创建线程并分离

关键代码：

```

1  BaseSock = socket(AF_INET, SOCK_STREAM, 0);
2  Addr.sin_family = AF_INET;
3  Addr.sin_addr.s_addr = INADDR_ANY;
4  Addr.sin_port = htons(6230); // 学号后四位
5  bind(BaseSock, reinterpret_cast<sockaddr*>(&Addr), sizeof(Addr));
6  listen(BaseSock, 10);
7
8  while (true) {
9      int client_sock = accept(BaseSock, nullptr, nullptr);
10     pthread_t tid;
11     pthread_create(&tid, nullptr, Recieve,
12                  reinterpret_cast<void*>((intptr_t)client_sock));
13     pthread_detach(tid);

```

```
14 }
```

3.2. Recieve 线程函数（服务端）

处理客户端请求的核心逻辑：

```
1  void* Recieve(void* lpParameter) { cpp
2      int sock = (int)(intptr_t)lpParameter;
3      char buffer[MAXBUF] = {0};
4      std::string recv_buffer;
5
6      while (true) {
7          ssize_t len = recv(sock, buffer, sizeof(buffer), 0);
8          if (len ≤ 0) { RemoveClient(sock); break; }
9
10         recv_buffer.append(buffer, len);
11         while (recv_buffer.size() ≥ 3) {
12             char type = recv_buffer[0];
13             uint16_t payload_len = ((uint8_t)recv_buffer[1] << 8) |
14                 (uint8_t)recv_buffer[2];
15             if (recv_buffer.size() < 3 + payload_len) break;
16
17             std::string payload = recv_buffer.substr(3, payload_len);
18             recv_buffer.erase(0, 3 + payload_len);
19
20             if (type == CONNECT) {
21                 // 分配新ID并返回
22                 int id = next_client_id++;
23                 clients[sock] = id;
24                 SendPacket(sock, CONNECT, std::to_string(id));
25             }
26             else if (type == TIME) {
27                 // 获取系统时间并返回
28                 std::time_t t = std::time(nullptr);
29                 std::string timestr = std::ctime(&t);
30                 SendPacket(sock, TIME, timestr);
31             }
32             // ... 其他类型处理
33         }
34     }
35     return nullptr;
36 }
```

3.3. 并发控制

使用互斥锁保护共享的 `clients` 容器：

```
1 pthread_mutex_t clients_mutex = PTHREAD_MUTEX_INITIALIZER;
2
3 // 访问clients前加锁
4 pthread_mutex_lock(&clients_mutex);
5 clients[sock] = id;
6 pthread_mutex_unlock(&clients_mutex);
```

这确保了多线程环境下的数据一致性。

4. 编译与运行

注意

实验文档中直接使用了 `g++` 进行编译，我这里为了方便、更规范，使用了 CMake 进行编译，构建脚本如下：

```
                                CMakeLists.txt
1 cmake_minimum_required (VERSION 3.20)
2 project (Lab6)
3
4 # generate compile_commands.json
5 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
6
7 add_executable(client
8     client/client.cpp
9     client/func.cpp
10 )
11
12 target_include_directories(client PRIVATE client/)
13 target_link_libraries(client PRIVATE pthread)
14
15 add_executable(server
16     server/server.cpp
17 )
18
19 target_include_directories(server PRIVATE server/)
20 target_link_libraries(server PRIVATE pthread)
```

在 `lab6/code` 目录下执行：

```
1 mkdir build
```

```
2 cd build
3 cmake ..
4 make -j
```

运行测试：

1. 启动服务器： `./server`
2. 新终端启动客户端： `./client`
3. 在客户端输入命令： `connect` , `time` , `list` , `message` , `disconnect` , `exit`

五、实验结果与分析

1. 协议报文格式

根据实验要求，我设计的应用层协议报文格式如 图 1

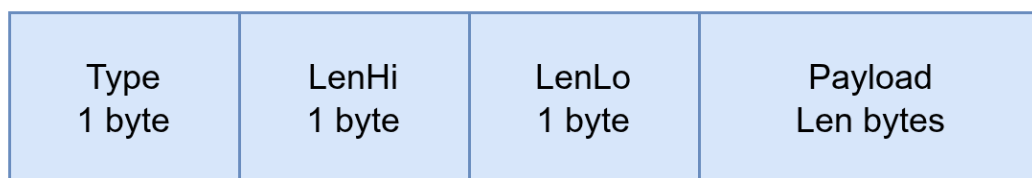


图 1: 报文格式示意图

请求和响应使用相同的报文格式，通过首字节的类型字段区分不同的操作。

2. 服务器启动

服务器启动后显示如 图 2

```
❖ → build git:(main) x ./server
[SRV] Server Building
[SRV] Server Build Successfully.
[SRV] Waiting for packs.
```

图 2: 服务器启动界面

服务器在端口 6230（学号后四位）上监听客户端连接请求。

3. 客户端启动

客户端启动后显示交互菜单，如 图 3

```

➔ build git:(main) ✕ ./client
[SYS] Client Initialization Now

[SYS] Nice to meet you, dear client! (^_~)
[SYS] Welcome to use my communication service.
[SYS] Please choose one from the following services:

[HELP]
connect Try to connect to the server.
exit Exit from this communication.
help Show all services for you.

[CLI] Your command: █

```

图 3: 客户端启动界面

4. 连接功能测试

客户端执行 `connect` 命令，服务端和客户端的输出结果如 图 4

```

➔ build git:(main) ✕ ./server
[SRV] Server Building
[SRV] Server Build Successfully.
[SRV] Waiting for packs.
[SRV] Client 1 connect successfully!
[SRV] Client ID returned.
█

➔ build git:(main) ✕ ./client
[SYS] Client Initialization Now

[SYS] Nice to meet you, dear client! (^_~)
[SYS] Welcome to use my communication service.
[SYS] Please choose one from the following services:

[HELP]
connect Try to connect to the server.
exit Exit from this communication.
help Show all services for you.

[CLI] Your command: connect
[SYS] Please enter the IP address and port you want to connect to.
[CLI] IP address: 127.0.0.1
[CLI] Port number: 6230
[SYS] Connecting ...
[SYS] Connection Success!

[CLI] Your command:
[CONNECT]
[SYS] Your Client ID is 1

[CLI] Your command: █

```

图 4: 连接功能测试

分析：客户端通过 TCP 三次握手成功连接到服务器，服务器为其分配唯一 ID（1）并通过 CONNECT 类型报文返回给客户端。

5. 获取时间功能测试

客户端执行 `time` 命令，服务端和客户端的输出结果如 图 5

```
+ build git:(main) ✗ ./server
[SRV] Server Building
[SRV] Server Build Successfully.
[SRV] Waiting for packs.
[SRV] Client 1 connect successfully!
[SRV] Client ID returned.
[SRV] Receive a TIME request from client 1
[SRV] Time Sending Back Successfully.
[]

+ build git:(main) ✗ ./client
[SYS] Client Initialization Now

[SYS] Nice to meet you, dear client! (^_^)
[SYS] Welcome to use my communication service.
[SYS] Please choose one from the following services:

[HELP]
connect Try to connect to the server.
exit Exit from this communication.
help Show all services for you.

[CLI] Your command: connect
[SYS] Please enter the IP address and port you want to connect to.
[CLI] IP address: 127.0.0.1
[CLI] Port number: 6230
[SYS] Connecting ...
[SYS] Connection Success!

[CLI] Your command:
[CONNECT]
[SYS] Your Client ID is 1

[CLI] Your command: time
[SYS] Request Sending Success

[CLI] Your command:
[TIME]
[SYS] Current Time is: Thu Dec 25 15:00:49 2025

[CLI] Your command: []
```

图 5: 获取时间功能测试

分析：客户端发送 TIME 类型请求（首字节为 3），服务器调用 `std::time()` 和 `std::ctime()` 获取系统时间并返回。

6. 获取主机名功能测试

客户端执行 `name` 命令，服务端和客户端的输出结果如 图 6

```
+ build git:(main) ✗ ./server
[SRV] Server Building
[SRV] Server Build Successfully.
[SRV] Waiting for packs.
[SRV] Client 1 connect successfully!
[SRV] Client ID returned.
[SRV] Receive a NAME request from client 1
[SRV] Name Sending Back Successfully.
[]

+ build git:(main) ✗ ./client
[SYS] Client Initialization Now

[SYS] Nice to meet you, dear client! (^_^)
[SYS] Welcome to use my communication service.
[SYS] Please choose one from the following services:

[HELP]
connect Try to connect to the server.
exit Exit from this communication.
help Show all services for you.

[CLI] Your command: connect
[SYS] Please enter the IP address and port you want to connect to.
[CLI] IP address: 127.0.0.1
[CLI] Port number: 6230
[SYS] Connecting ...
[SYS] Connection Success!

[CLI] Your command:
[CONNECT]
[SYS] Your Client ID is 1

[CLI] Your command: name
[SYS] Request Sending Success

[CLI] Your command:
[NAME]
[SYS] Server Hostname is: cs144
```

图 6: 获取主机名功能测试

分析：服务器通过 `gethostname()` 系统调用获取本机主机名并返回。

7. 获取客户端列表功能测试

启动第二个客户端（ID 为 2），然后在客户端 1 执行 `list` 命令，服务端和客户端的输出结果如 图 7

```
+ build git:(main) ✖ ./server
[SRV] Server Building
[SRV] Server Build Successfully.
[SRV] Waiting for packs.
[SRV] Client 1 connect successfully!
[SRV] Client ID returned.
[SRV] Receive a NAME request from client 1
[SRV] Name Sending Back Successfully.
[SRV] Client 2 connect successfully!
[SRV] Client ID returned.
[SRV] Receive a LIST request from client 1
[SRV] List Sending Back Successfully.
[]

connect Try to connect to the server.
exit Exit from this communication.
help Show all services for you.

[CLI] Your command: connect
[SYS] Please enter the IP address and port you want to connect to.
[CLI] IP address: 127.0.0.1
[CLI] Port number: 6230
[SYS] Connecting ...
[SYS] Connection Success!

[CLI] Your command:
[CONNECT]
[SYS] Your Client ID is 1

[CLI] Your command: name
[SYS] Request Sending Success

[CLI] Your command:
[NAME]
[SYS] Server Hostname is: cs144

[CLI] Your command: list
[SYS] Request Sending Success

[CLI] Your command:
[LIST]
[SYS] Client List: 1$2

+ build git:(main) ✖ ./client
[SYS] Client Initialization Now

[SYS] Nice to meet you, dear client! (^~)
[SYS] Welcome to use my communication service.
[SYS] Please choose one from the following services:

[HELP]
connect Try to connect to the server.
exit Exit from this communication.
help Show all services for you.

[CLI] Your command: connect
[SYS] Please enter the IP address and port you want to connect to.
[CLI] IP address: [CLI] Port number: 6230
[SYS] Connecting ...
[SYS] Connection Success!

[CLI] Your command:
[CONNECT]
[SYS] Your Client ID is 2

[CLI] Your command:
[CONNECT]
[SYS] Your Client ID is 2

[CLI] Your command: []
```

图 7: 获取客户端列表功能测试

分析：服务器遍历 `clients` 映射表，将所有在线客户端 ID 用 `$` 分隔符连接成字符串返回。互斥锁保证了并发访问的安全性。

8. 消息转发功能测试

假设有客户端 1 和客户端 2，客户端 2 向客户端 1 发送消息，执行 `message` 命令，服务端和客户端的输出结果如图 8

```
+ build git:(main) ✖ ./server
[SRV] Server Building
[SRV] Server Build Successfully.
[SRV] Waiting for packs.
[SRV] Client 1 connect successfully!
[SRV] Client ID returned.
[SRV] Client 2 connect successfully!
[SRV] Client ID returned.
[SRV] Receive a MESSAGE request from client 2
[SRV] Message Sending Success
[]

+ build git:(main) ✖ ./client
[SYS] Client Initialization Now

[SYS] Nice to meet you, dear client! (^~)
[SYS] Welcome to use my communication service.
[SYS] Please choose one from the following services:

[HELP]
connect Try to connect to the server.
exit Exit from this communication.
help Show all services for you.

[CLI] Your command: connect
[SYS] Please enter the IP address and port you want to connect to.
[CLI] IP address: 127.0.0.1
[CLI] Port number: 6230
[SYS] Connecting ...
[SYS] Connection Success!

[CLI] Your command:
[CONNECT]
[SYS] Your Client ID is 1

[CLI] Your command:
[RECEIVE MESSAGE]
[SYS] Client 2 Just Sent A Message to you:
Hello, world!

[CLI] Your command: []

[SYS] Please choose one from the following services:

[HELP]
connect Try to connect to the server.
exit Exit from this communication.
help Show all services for you.

[CLI] Your command: connect
[SYS] Please enter the IP address and port you want to connect to.
[CLI] IP address: 127.0.0.1
[CLI] Port number: 6230
[SYS] Connecting ...
[SYS] Connection Success!

[CLI] Your command:
[CONNECT]
[SYS] Your Client ID is 2

[CLI] Your command: message
[SYS] Please Enter Other Clients' ID and the Message (on a line)
[CLI] Client ID: 1
[CLI] Message: Hello, world!
[SYS] Message Sending Success.

[CLI] Your command:
[MESSAGE]
[SYS] Server Reply: Message Sending Success

[CLI] Your command: []
```

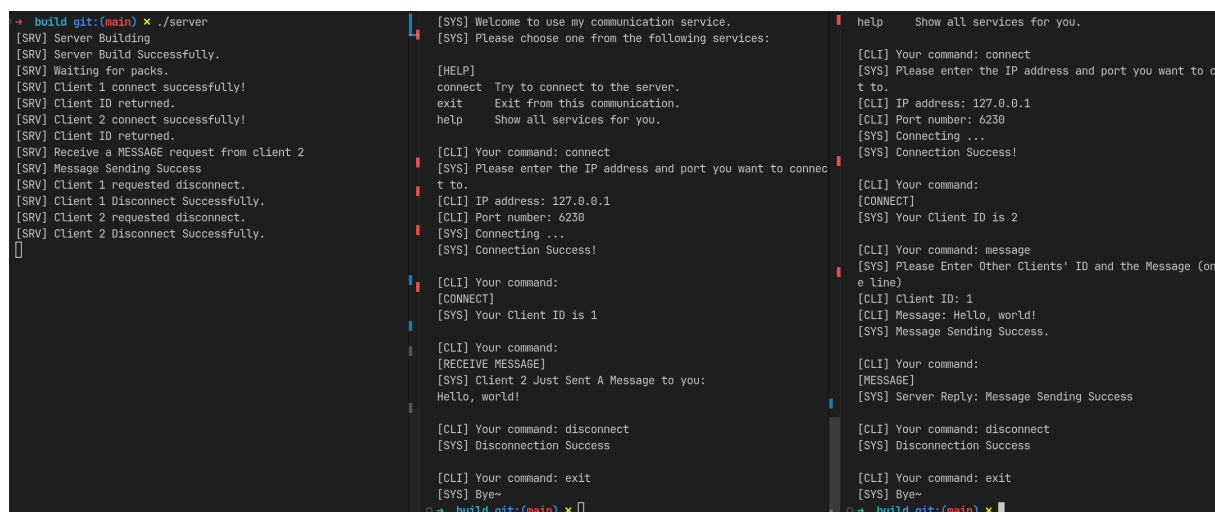
图 8: 消息转发功能测试

分析：

1. 客户端 2 发送 MESSAGE 报文，负载格式为 `"1$Hello, world!"`
2. 服务器解析目标 ID (1)，在 `clients` 中查找对应 socket
3. 服务器向客户端 1 转发 SIGNAL 报文。
4. 客户端 1 的接收线程解析 SIGNAL 报文并显示。

9. 断开连接功能测试

客户端执行 `disconnect` 命令，服务端和客户端的输出结果如图 9



The image shows three terminal windows side-by-side. The left window shows the server's output, including messages like 'Server Build Successfully', 'Client 1 connect successfully!', and 'Client 1 Disconnect Successfully.'. The middle window shows the help menu and the process of connecting and then disconnecting Client 1. The right window shows the process of connecting and then disconnecting Client 2. All windows show the successful execution of the 'disconnect' command.

图 9: 断开连接功能测试

分析：客户端发送 DISCONNECT 报文后关闭 socket，服务器收到后调用 `RemoveClient()` 从 `clients` 映射表中删除该客户端并关闭对应 socket。

10. 并发性能测试

10.1. 测试 1：单客户端连续请求

修改客户端代码 `client/client.cpp` 中的 `GetTime()`，自动发送 100 次请求并计数响应。

```
client/func.cpp  cpp
1 void GetTime() {
2     if (!connected) { std::cout << "[SYS] Not connected." << std::endl;
3         return; }
4     for (int i = 0; i < 100; ++i) {
5         if (SendPacket(TIME, "")) std::cout << "[SYS] Request Sending
6             Success" << std::endl;
7     }
8 }
9 // ...
10 void* Recieve(void* lpParameter) {
11     // ...
12 }
13 while (connected) {
14     // ...
```


10.2. 测试 2：多客户端并发请求

同时启动 3 个客户端，每个客户端连续发送 100 次 TIME 请求。我创建了一个简单的脚本 `concurrent.sh` 来自动化测试：

```
concurrent.sh sh
1  #!/bin/bash
2
3  # 默认值
4  CLIENT_EXEC="./client/client.out"
5  CLIENT_COUNT=2
6
7  # 解析参数
8  if [ "$#" -ge 1 ]; then
9      CLIENT_EXEC=$1
10 fi
11
12 if [ "$#" -ge 2 ]; then
13     CLIENT_COUNT=$2
14 fi
15
16 # 检查可执行文件是否存在
17 if [ ! -f "$CLIENT_EXEC" ]; then
18     echo "Error: $CLIENT_EXEC not found. Please compile first or provide
19     correct path."
20     echo "Usage: $0 [client_path] [client_count]"
21     exit 1
22 fi
23 run_client_input() {
24     echo "connect"
25     sleep 0.5
26     echo "127.0.0.1"
27     echo "6230"
28
29     sleep 1      # 等待连接建立
30     echo "time"  # 发送请求（触发100次发送）
31
32     sleep 3      # 等待接收所有响应
33     echo "exit"
34 }
35
36 echo "≡ Starting Concurrent Test ≡"
37 echo "Client Path: $CLIENT_EXEC"
```

```

38 echo "Client Count: $CLIENT_COUNT"
39 echo "Make sure your server is running in another terminal!"
40
41 PIDS=""
42
43 for ((i=1; i<=CLIENT_COUNT; i++)); do
44     LOG_FILE="client_${i}.log"
45     echo "[TEST] Launching Client $i..."
46     run_client_input | $CLIENT_EXEC > "$LOG_FILE" 2>&1 &
47     PIDS="$PIDS $!"
48 done
49
50 echo "[TEST] Clients running with PIDs: $PIDS"
51 echo "[TEST] Waiting for tests to complete..."
52
53 # 等待所有进程结束
54 for pid in $PIDS; do
55     wait $pid
56 done
57
58 echo "=== Test Finished ==="
59 echo "Check the server terminal for concurrent logs."
60
61 # 显示每个客户端日志的最后几行
62 for ((i=1; i<=CLIENT_COUNT; i++)); do
63     LOG_FILE="client_${i}.log"
64     echo -e "\n--- Client $i Log (Last 5 lines) ---"
65     tail -n 5 "$LOG_FILE"
66 done

```

先执行服务器端程序，然后运行脚本：

```
1 ./concurrent.sh ./build/client 3
```

sh

服务端和客户端的输出如图 11

```
[SRV] Time Sending Back Successfully.
[SRV] Receive a TIME request from client 3
[SRV] Time Sending Back Successfully.
[SRV] Receive a TIME request from client 3
[SRV] Time Sending Back Successfully.
[SRV] Receive a TIME request from client 3
[SRV] Time Sending Back Successfully.
[SRV] Receive a TIME request from client 3
[SRV] Time Sending Back Successfully.
[SRV] Receive a TIME request from client 3
[SRV] Time Sending Back Successfully.
[SRV] Receive a TIME request from client 3
[SRV] Time Sending Back Successfully.
[SRV] Receive a TIME request from client 3
[SRV] Time Sending Back Successfully.
[SRV] Receive a TIME request from client 3
[SRV] Time Sending Back Successfully.
[SRV] Receive a TIME request from client 3
[SRV] Time Sending Back Successfully.
[SRV] Client 1 requested disconnect.
[SRV] Client 1 Disconnect Successfully.
[SRV] Client 3 requested disconnect.
[SRV] Client 3 Disconnect Successfully.
[SRV] Client 2 requested disconnect.
[SRV] Client 2 Disconnect Successfully.

code git:(main) ✖ ./concurrent.sh ./build/client 3
=== Starting Concurrent Test ===
Client Path: ./build/client
Client Count: 3
Make sure your server is running in another terminal!
[TEST] Launching Client 1...
[TEST] Launching Client 2...
[TEST] Launching Client 3...
[TEST] Clients running with PIDs: 88021 88024 88028
[TEST] Waiting for tests to complete...
=== Test Finished ===
Check the server terminal for concurrent logs.

--- Client 1 Log (Last 5 lines) ---
[TIME]
[SYS] Current Time is: Thu Dec 25 15:35:16 2025, 100 responses received.
[CLI] Your command: [SYS] Disconnection Success
[SYS] Bye~

--- Client 2 Log (Last 5 lines) ---
[TIME]
[SYS] Current Time is: Thu Dec 25 15:35:16 2025, 100 responses received.
[CLI] Your command: [SYS] Disconnection Success
[SYS] Bye~
```

图 11: 多客户端并发请求测试

分析：服务器成功处理了 3 个客户端的并发 TIME 请求，每个客户端均正确接收了 100 次响应，验证了服务器的并发处理能力。

六、思考题

1. 问题一

客户端是否需要调用 `bind` 操作？它的源端口是如何产生的？每一次调用 `connect` 时客户端的端口是否都保持不变？

客户端通常不需要调用 `bind` 操作。当客户端调用 `connect()` 时，如果 `socket` 尚未绑定到本地地址，操作系统内核会自动为其分配一个临时端口（ephemeral port），端口范围通常在 32768-60999 之间（可通过 `/proc/sys/net/ipv4/ip_local_port_range` 查看）。

每次调用 `connect` 时，如果没有显式 `bind`，客户端的源端口通常会变化。这是因为如果前一个连接已关闭，旧的 `socket` 会进入 `TIME_WAIT` 状态，该端口暂时不可用，操作系统会从可用端口池中选择一个新的端口分配给新 `socket`。只有在特殊场景下（如需要固定源端口、或需要连接同一服务器的相同端口多次），客户端才会显式调用 `bind`。

2. 问题二

假设在服务端调用 `listen` 和调用 `accept` 之间设了一个调试断点，暂停在此断点时，此时客户端调用 `connect` 后是否马上能连接成功？

可以连接成功，但是应用程序需要等待断点继续执行后才能处理连接。

当服务器调用 `listen()` 后，内核会维护两个队列：半连接队列（SYN queue）存放收到 SYN 但未完成三次握手的连接，全连接队列（accept queue）存放已完成三次握手但尚未被 `accept()` 取走的连接。

即使在 `listen()` 和 `accept()` 之间暂停，客户端的 `connect()` 仍可以完成三次握手（因为握手由内核处理），连接会被放入全连接队列。此时客户端的 `connect()` 会返回成功，连接在 TCP 层面已建立，可以发送数据。只有当程序从断点继续执行并调用 `accept()` 后，应用程序才能获取这个连接并开始处理。因此，断点不会阻止 TCP 连接的建立，但会延迟应用层对连接的处理。

3. 问题三

服务器在同一个端口接收多个客户端的数据，如何能区分数据包是属于哪个客户端的？

服务器通过不同的 socket 文件描述符来区分不同客户端。

服务器创建一个监听 socket 并绑定到特定端口（如 6230），这个 socket 只用于监听连接请求，不用于数据传输。每当 `accept()` 接受一个新连接时，内核会创建一个新的 socket（返回新的文件描述符），这个 socket 代表与特定客户端的连接。

每个 TCP 连接由五元组唯一标识：（服务器IP，服务器端口，客户端IP，客户端端口，协议）。虽然服务器 IP 和端口相同，但每个客户端的 IP 或端口至少有一个不同。

在我的实现中，使用 `std::map<int, int> clients` 将 socket 描述符映射到客户端 ID：

```
1 int client_sock = accept(BaseSock, nullptr, nullptr);  
2 clients[client_sock] = client_id; // 每个socket对应一个客户端
```

当 `recv(client_sock, ...)` 接收数据时，通过 `client_sock` 就能知道数据来自哪个客户端。

4. 问题四

客户端主动断开连接后，当时的 TCP 连接状态是什么？这个状态保持了多久？（可以使用 `netstat -an` 查看）

客户端主动断开连接（调用 `close()`）后，会经历以下状态变化：

1. **FIN_WAIT_1**: 客户端发送 FIN 包后立即进入此状态
2. **FIN_WAIT_2**: 收到服务器的 ACK 后进入此状态

3. TIME_WAIT: 收到服务器的 FIN 并发送 ACK 后进入此状态

可以通过 `netstat -an` 验证:

```
1 netstat -an | grep 6230 bash
2 # 可以看到这一行:
3 tcp 0 0 127.0.0.1:45678 127.0.0.1:6230 TIME_WAIT
```

TIME_WAIT 状态存在的原因是为了确保可靠关闭 (确保最后的 ACK 能到达服务器) 以及防止旧连接干扰 (确保旧连接的延迟数据包不会被新连接误收)。在 TIME_WAIT 期间, 该 (客户端IP:端口, 服务器IP:端口) 四元组不能被重用。

5. 问题五

客户端断网后异常退出, 服务器的 TCP 连接状态有什么变化吗? 服务器该如何检测连接是否继续有效?

客户端异常断网时, 服务器的 TCP 连接会保持在 ESTABLISHED 状态, 因为没有收到 FIN 包。如果服务器向客户端发送数据, 会发生超时重传, 多次重传失败后, TCP 会报告连接错误。如果服务器不主动发送数据, 可能永远检测不到断开, 长时间保持无效连接会占用系统资源。

我的实现通过检查 `recv()` 的返回值来检测连接状态。需要注意的是, 这种方法只能检测到对端进程正常退出 (发送 FIN) 或主机崩溃重启 (发送 RST) 的情况。对于直接断网或断电这种“静默”死连接, `recv()` 会一直阻塞, 服务器无法立即感知, 必须引入心跳机制或 TCP Keepalive 才能解决。

七、讨论、心得

通过本次实验, 我深入理解了 Socket 编程和网络应用协议的设计与实现。本次实验是从应用层角度使用 TCP 协议, 与之前实验中实现 TCP 协议栈形成了完整的认知闭环。

1. 主要收获

首先, 我对 Socket API 有了更深刻的理解。掌握了 `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()` 等系统调用的用法和时机, 理解了面向连接的 TCP 通信流程。

其次, 积累了协议设计的经验。学会了如何设计简洁高效的应用层协议。通过“类型字节+负载”的格式, 用最少的开销实现了多种功能的区分。使用 `$` 作为分隔符来传递结构化数据 (如消息转发中的 `id$内容`), 简单且有效。

在多线程并发编程方面，我实践了服务器端使用“one thread per connection”模型处理并发，客户端使用独立接收线程避免阻塞用户交互。同时，学会了使用 `pthread_mutex_t` 保护共享数据结构 `cClients`，并理解了 `pthread_detach()` 的作用：分离线程以自动回收资源。

最后，在网络编程实践中，我理解了监听 socket 和连接 socket 的区别，掌握了 TCP 连接的四次挥手和 `TIME_WAIT` 状态，学会了通过 socket 文件描述符区分不同客户端，并认识到了连接异常处理的重要性（如客户端断网检测）。

2. 遇到的困难与解决

在实验过程中，我遇到了一些困难。例如，客户端发送消息时，如何正确读取包含空格的完整消息行？我通过先用 `std::cin >> id` 读取 ID，然后调用 `std::cin.ignore()` 清空输入缓冲区中的换行符，最后用 `std::getline()` 读取完整消息来解决。

另一个问题是服务器在多线程环境下访问 `cClients` 映射时偶尔崩溃。通过在所有访问 `cClients` 的地方添加互斥锁保护，避免了数据竞争导致的未定义行为。

此外，客户端接收线程与主线程的输出交错，导致提示符显示混乱。我在接收线程打印消息后，主动调用 `PrintPrompt()` 重新显示命令提示符，提升了用户体验。

在错误处理方面，当前实现对网络异常的处理较简单，生产环境需要更完善的错误恢复机制。最后是安全性，明文传输存在安全隐患，可以考虑使用 TLS/SSL 加密通信。本次实验让我对网络编程有了更深入的理解，也认识到了从协议设计到实现的完整流程。这些知识和经验将为今后的网络应用开发打下坚实基础。

八、附录

1. client/client.cpp

```
client/client.cpp  cpp
1  #include "client.h"
2  #include "func.h"
3
4  bool connected = false;
5  int BaseSock = -1;
6  sockaddr_in Addr {};
7  pthread_t thread {};
8
9  // 显示帮助菜单
10 void Help(bool wrong) {
```

```

11     if (wrong) {
12         std::cout << "\n[SYS] Sorry, your input service is illegal.
           Please choose one from following services:" << std::endl;
13     }
14
15     std::cout << "\n[HELP]" << std::endl;
16     std::cout << "connect\t Try to connect to the server." << std::endl;
17
18     if (connected) {
19         std::cout << "disconnect\t Disconnect from the server." <<
           std::endl;
20         std::cout << "time\t Get current time." << std::endl;
21         std::cout << "name\t Get name of machine." << std::endl;
22         std::cout << "list\t Get list of all clients." << std::endl;
23         std::cout << "message\t Send message to other clients." <<
           std::endl;
24     }
25
26     std::cout << "exit\t Exit from this communication." << std::endl;
27     std::cout << "help\t Show all services for you." << std::endl;
28 }
29
30 void Initialize() {
31     std::cout << "[SYS] Client Initialization Now" << std::endl <<
       std::endl;
32     std::cout << "[SYS] Nice to meet you, dear client! (^_~)" <<
       std::endl;
33     std::cout << "[SYS] Welcome to use my communication service." <<
       std::endl;
34     std::cout << "[SYS] Please choose one from the following services:"
       << std::endl;
35
36     Help(false);
37
38     std::string command;
39
40     while (true) {
41         PrintPrompt();
42         if (!(std::cin >> command)) {
43             break;
44         }
45
46         if (command == "help") {

```

```

47         Help(false);
48     } else if (command == "connect") {
49         ServerConnect();
50     } else if (command == "exit") {
51         ExitComm();
52         break;
53     } else if (!connected) {
54         Help(true);
55     } else if (command == "disconnect") {
56         ServerDisconnect();
57     } else if (command == "time") {
58         GetTime();
59     } else if (command == "name") {
60         GetName();
61     } else if (command == "list") {
62         ClientList();
63     } else if (command == "message") {
64         SendMessa();
65     } else {
66         Help(true);
67     }
68 }
69 }
70
71 int main() {
72     Initialize();
73 }

```

2. client/func.cpp

client/func.cpp		cpp
1	#include "func.h"	
2	#include <limits>	
3		
4	namespace {	
5		
6	bool SendPacket(char type, const std::string& payload = "") {	
7	if (!connected BaseSock < 0) return false;	
8		
9	uint16_t payload_len = payload.size();	
10	std::string packet;	
11	packet.reserve(1 + 2 + payload_len);	
12	packet.push_back(type);	// 类型(1字节)

```

13     packet.push_back((payload_len >> 8) & 0xFF); // 长度高字节
14     packet.push_back(payload_len & 0xFF);        // 长度低字节
15     packet.append(payload);                       // 负载数据
16
17     ssize_t sent = send(BaseSock, packet.data(), packet.size(), 0);
18     return sent == (ssize_t)packet.size();
19 }
20
21 } // namespace
22
23 void PrintPrompt() {
24     std::cout << "\n[CLI] Your command: " << std::flush;
25 }
26
27 void ServerConnect() {
28     if (connected) {
29         std::cout << "[SYS] Already connected." << std::endl;
30         return;
31     }
32
33     std::string ip;
34     int port;
35     std::cout << "[SYS] Please enter the IP address and port you want to
36     connect to." << std::endl;
37     std::cout << "[CLI] IP address: ";
38     std::cin >> ip;
39     std::cout << "[CLI] Port number: ";
40     std::cin >> port;
41
42     std::cout << "[SYS] Connecting ..." << std::endl;
43
44     int sock = socket(AF_INET, SOCK_STREAM, 0);
45     if (sock < 0) {
46         std::perror("socket");
47         return;
48     }
49
50     Addr.sin_family = AF_INET;
51     Addr.sin_port = htons(port);
52     if (inet_pton(AF_INET, ip.c_str(), &Addr.sin_addr) <= 0) {
53         std::cerr << "[SYS] Invalid IP address." << std::endl;
54         close(sock);
55         return;

```

```

55     }
56
57     if (connect(sock, reinterpret_cast<sockaddr*>(&Addr), sizeof(Addr))
58         < 0) {
59         std::perror("connect");
60         close(sock);
61         return;
62     }
63     BaseSock = sock;
64     connected = true;
65     std::cout << "[SYS] Connection Success!" << std::endl;
66
67     SendPacket(CONNECT, "");
68
69     // start receive thread
70     if (pthread_create(&thread, nullptr, Recieve, nullptr) != 0) {
71         std::cerr << "[SYS] Failed to start receive thread." <<
72         std::endl;
73         // still connected, but no receive thread
74     } else {
75         pthread_detach(thread);
76     }
77
78     void ServerDisconnect() {
79         if (!connected) return;
80         SendPacket(DISCONNECT, "");
81         connected = false;
82         close(BaseSock);
83         BaseSock = -1;
84         std::cout << "[SYS] Disconnection Success" << std::endl;
85     }
86
87     void GetTime() {
88         if (!connected) { std::cout << "[SYS] Not connected." << std::endl;
89         return; }
90         for (int i = 0; i < 100; ++i) {
91             if (SendPacket(TIME, "")) std::cout << "[SYS] Request Sending
92             Success" << std::endl;
93         }
94     }
95 }

```

```

94 void GetName() {
95     if (!connected) { std::cout << "[SYS] Not connected." << std::endl;
        return; }
96     if (SendPacket(NAME, "")) std::cout << "[SYS] Request Sending
        Success" << std::endl;
97 }
98
99 void ClientList() {
100     if (!connected) { std::cout << "[SYS] Not connected." << std::endl;
        return; }
101     if (SendPacket(LIST, "")) std::cout << "[SYS] Request Sending
        Success" << std::endl;
102 }
103
104 void SendMessa() {
105     if (!connected) { std::cout << "[SYS] Not connected." << std::endl;
        return; }
106
107     std::string id;
108     std::string msg;
109     std::cout << "[SYS] Please Enter Other Clients' ID and the Message
        (only one line)" << std::endl;
110     std::cout << "[CLI] Client ID: ";
111     std::cin >> id;
112     std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
113     std::cout << "[CLI] Message: ";
114     std::getline(std::cin, msg);
115     std::string payload = id + "$" + msg;
116     if (SendPacket(MESSAGE, payload)) std::cout << "[SYS] Message
        Sending Success." << std::endl;
117 }
118
119 void ExitComm() {
120     if (connected) {
121         ServerDisconnect();
122     }
123     std::cout << "[SYS] Bye~" << std::endl;
124 }
125
126 void* Recieve(void* lpParameter) {
127     (void)lpParameter;
128     char buffer[MAXBUF] = {0};
129     std::string recv_buffer; // 接收缓冲区

```

```

130
131     while (connected) {
132         ssize_t len = recv(BaseSock, buffer, sizeof(buffer), 0);
133         if (!connected) break;
134         if (len ≤ 0) {
135             std::cout << "[SYS] Server disconnected or error." <<
                std::endl;
136             connected = false;
137             if (BaseSock ≥ 0) close(BaseSock);
138             BaseSock = -1;
139             break;
140         }
141
142         recv_buffer.append(buffer, len);
143
144         // 循环处理缓冲区中的完整消息
145         while (recv_buffer.size() ≥ 3) { // 至少需要Type(1) + Length(2)
146             char type = recv_buffer[0];
147             uint16_t payload_len = ((uint8_t)recv_buffer[1] << 8) |
                (uint8_t)recv_buffer[2];
148
149             if (recv_buffer.size() < 3 + payload_len) {
150                 break; // 数据不完整, 等待下次recv
151             }
152
153             std::string payload = recv_buffer.substr(3, payload_len);
154             recv_buffer.erase(0, 3 + payload_len); // 移除已处理的数据
155
156             static int time_recv_cnt = 0;
157
158             switch (type) {
159                 case CONNECT:
160                     std::cout << "\n[CONNECT]" << std::endl;
161                     std::cout << "[SYS] Your Client ID is " << payload <<
                        std::endl;
162                     break;
163                 case TIME:
164                     std::cout << "\n[TIME]" << std::endl;
165                     std::cout << "[SYS] Current Time is: " << payload << ",
                        " << ++time_recv_cnt << " responses received." <<
                        std::endl;
166                     break;
167                 case NAME:

```

```

168         std::cout << "\n[NAME]" << std::endl;
169         std::cout << "[SYS] Server Hostname is: " << payload <<
            std::endl;
170         break;
171     case LIST:
172         std::cout << "\n[LIST]" << std::endl;
173         std::cout << "[SYS] Client List: " << payload <<
            std::endl;
174         break;
175     case MESSAGE:
176         std::cout << "\n[MESSAGE]" << std::endl;
177         std::cout << "[SYS] Server Reply: " << payload <<
            std::endl;
178         break;
179     case SIGNAL: {
180         std::cout << "\n[RECEIVE MESSAGE]" << std::endl;
181         // payload expected: srcid$contents
182         auto pos = payload.find('$');
183         if (pos != std::string::npos) {
184             std::string src = payload.substr(0, pos);
185             std::string msg = payload.substr(pos + 1);
186             std::cout << "[SYS] Client " << src << " Just Sent A
                Message to you:\n";
187             std::cout << msg << std::endl;
188         } else {
189             std::cout << "[SYS] Received signal: " << payload <<
                std::endl;
190         }
191         break;
192     }
193     case INVALID:
194     default:
195         std::cout << "\n[INVALID] " << payload << std::endl;
196         break;
197     }
198     PrintPrompt();
199 } // end while processing messages in recv_buffer
200 }
201 return nullptr;
202 }

```

3. server/server.cpp

```
server/server.cpp  cpp
1  #include "server.h"
2
3  // 全局变量初始化
4  int BaseSock = -1;
5  sockaddr_in Addr {};
6  // 请填入你的学号后四位作为端口号
7  u_short Port = 6230;
8  std::map<int, int> clients;
9  pthread_mutex_t clients_mutex = PTHREAD_MUTEX_INITIALIZER;
10
11 static int next_client_id = 1;
12
13 // 发送数据包
14 bool SendPacket(int sock, char type, const std::string& payload) {
15     if (sock < 0) return false;
16
17     uint16_t payload_len = payload.size();
18     std::string packet;
19     packet.reserve(1 + 2 + payload_len);
20     packet.push_back(type); // 类型(1字节)
21     packet.push_back((payload_len >> 8) & 0xFF); // 长度高字节
22     packet.push_back(payload_len & 0xFF); // 长度低字节
23     packet.append(payload); // 负载数据
24
25     ssize_t s = send(sock, packet.data(), packet.size(), 0);
26     return s == (ssize_t)packet.size();
27 }
28
29 void RemoveClient(int sock) {
30     pthread_mutex_lock(&clients_mutex);
31     auto it = clients.find(sock);
32     if (it != clients.end()) {
33         int id = it->second;
34         clients.erase(it);
35         std::cout << "[SRV] Client " << id << " Disconnect  
Successfully." << std::endl;
36     } else {
37         std::cout << "[SRV] Unknown client socket disconnected: " <<  
sock << std::endl;
38     }
```

```

39     pthread_mutex_unlock(&clients_mutex);
40     if (sock ≥ 0) close(sock);
41 }
42
43 void BuildServer() {
44     std::cout << "[SRV] Server Building" << std::endl;
45     BaseSock = socket(AF_INET, SOCK_STREAM, 0);
46     if (BaseSock < 0) {
47         std::perror("socket");
48         return;
49     }
50
51     Addr.sin_family = AF_INET;
52     Addr.sin_addr.s_addr = INADDR_ANY;
53     Addr.sin_port = htons(Port);
54
55     int opt = 1;
56     setsockopt(BaseSock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
57
58     if (bind(BaseSock, reinterpret_cast<sockaddr*>(&Addr), sizeof(Addr))
59         < 0) {
60         std::perror("bind");
61         close(BaseSock);
62         return;
63     }
64
65     if (listen(BaseSock, 10) < 0) {
66         std::perror("listen");
67         close(BaseSock);
68         return;
69     }
70
71     std::cout << "[SRV] Server Build Successfully." << std::endl;
72     std::cout << "[SRV] Waiting for packs." << std::endl;
73
74     while (true) {
75         int client_sock = accept(BaseSock, nullptr, nullptr);
76         if (client_sock < 0) {
77             std::perror("accept");
78             continue;
79         }
80
81         pthread_mutex_lock(&clients_mutex);

```

```

81     clients[client_sock] = 0; // placeholder until CONNECT
82     pthread_mutex_unlock(&clients_mutex);
83
84     pthread_t tid;
85     intptr_t arg = (intptr_t)client_sock;
86     if (pthread_create(&tid, nullptr, Recieve,
87         reinterpret_cast<void*>(arg)) != 0) {
87         std::cerr << "[SRV] Failed to create thread for client" <<
            std::endl;
88         RemoveClient(client_sock);
89     } else {
90         pthread_detach(tid);
91     }
92 }
93 }
94
95 void* Recieve(void* lpParameter) {
96     int sock = (int)(intptr_t)lpParameter;
97     char buffer[MAXBUF] = {0};
98     std::string recv_buffer; // 接收缓冲区, 处理粘包/拆包
99
100     while (true) {
101         ssize_t len = recv(sock, buffer, sizeof(buffer), 0);
102         if (len <= 0) {
103             RemoveClient(sock);
104             break;
105         }
106
107         recv_buffer.append(buffer, len);
108
109         // 循环处理缓冲区中的完整消息
110         while (recv_buffer.size() >= 3) { // 至少需要Type(1) + Length(2)
111             char type = recv_buffer[0];
112             uint16_t payload_len = ((uint8_t)recv_buffer[1] << 8) |
                (uint8_t)recv_buffer[2];
113
114             if (recv_buffer.size() < 3 + payload_len) {
115                 break; // 数据不完整, 等待下次recv
116             }
117
118             std::string payload = recv_buffer.substr(3, payload_len);
119             recv_buffer.erase(0, 3 + payload_len); // 移除已处理的数据
120

```

```

121         if (type == CONNECT) {
122             pthread_mutex_lock(&clients_mutex);
123             int id = next_client_id++;
124             clients[sock] = id;
125             pthread_mutex_unlock(&clients_mutex);
126             std::cout << "[SRV] Client " << id << " connect
127             successfully!" << std::endl;
128             SendPacket(sock, CONNECT, std::to_string(id));
129             std::cout << "[SRV] Client ID returned." << std::endl;
130         } else if (type == TIME) {
131             pthread_mutex_lock(&clients_mutex);
132             int id = clients[sock];
133             pthread_mutex_unlock(&clients_mutex);
134             std::cout << "[SRV] Receive a TIME request from client " <<
135             id << std::endl;
136             std::time_t t = std::time(nullptr);
137             std::string timestr = std::ctime(&t);
138             if (!timestr.empty() && timestr.back() == '\n')
139                 timestr.pop_back();
140             SendPacket(sock, TIME, timestr);
141             std::cout << "[SRV] Time Sending Back Successfully." <<
142             std::endl;
143         } else if (type == NAME) {
144             pthread_mutex_lock(&clients_mutex);
145             int id = clients[sock];
146             pthread_mutex_unlock(&clients_mutex);
147             std::cout << "[SRV] Receive a NAME request from client " <<
148             id << std::endl;
149             char host[256] = {0};
150             gethostname(host, sizeof(host));
151             SendPacket(sock, NAME, std::string(host));
152             std::cout << "[SRV] Name Sending Back Successfully." <<
153             std::endl;
154         } else if (type == LIST) {
155             pthread_mutex_lock(&clients_mutex);
156             int id = clients[sock];
157             std::cout << "[SRV] Receive a LIST request from client " <<
158             id << std::endl;
159             std::string list;
160             bool first = true;
161             for (const auto &p : clients) {
162                 int cid = p.second;
163                 if (cid <= 0) continue;

```

```

157         if (!first) list.push_back('$');
158         list.append(std::to_string(cid));
159         first = false;
160     }
161     pthread_mutex_unlock(&clients_mutex);
162     SendPacket(sock, LIST, list);
163     std::cout << "[SRV] List Sending Back Successfully." <<
        std::endl;
164 } else if (type == MESSAGE) {
165     // payload: targetid$content
166     pthread_mutex_lock(&clients_mutex);
167     int srcid = clients[sock];
168     pthread_mutex_unlock(&clients_mutex);
169     std::cout << "[SRV] Receive a MESSAGE request from client "
        << srcid << std::endl;
170     auto pos = payload.find('$');
171     if (pos == std::string::npos) {
172         SendPacket(sock, INVALID, "Bad message format");
173         continue;
174     }
175     std::string target = payload.substr(0, pos);
176     std::string content = payload.substr(pos + 1);
177     int target_id = std::stoi(target);
178
179     int target_sock = -1;
180     pthread_mutex_lock(&clients_mutex);
181     for (const auto &p : clients) {
182         if (p.second == target_id) { target_sock = p.first;
            break; }
183     }
184     pthread_mutex_unlock(&clients_mutex);
185
186     if (target_sock < 0) {
187         SendPacket(sock, INVALID, "Target not found");
188     } else {
189         // forward as SIGNAL with payload srcid$content
190         std::string forward = std::to_string(srcid) + "$" +
            content;
191         SendPacket(target_sock, SIGNAL, forward);
192         SendPacket(sock, MESSAGE, "Message Sending Success");
193         std::cout << "[SRV] Message Sending Success" <<
            std::endl;
194     }

```

```

195         } else if (type == DISCONNECT) {
196             pthread_mutex_lock(&clients_mutex);
197             int id = clients[sock];
198             pthread_mutex_unlock(&clients_mutex);
199             std::cout << "[SRV] Client " << id << " requested
200             disconnect." << std::endl;
201             RemoveClient(sock);
202             return nullptr;
203         } else {
204             SendPacket(sock, INVALID, "Unsupported type");
205         } // end while processing messages in recv_buffer
206     }
207     return nullptr;
208 }
209
210 int main() {
211     BuildServer();
212 }

```